EVM 모듈 구조 분석

1. EVM 모듈 개요

EVM 모듈(`x/evm`)은 MiniEVM의 핵심 컴포넌트로, Ethereum Virtual Machine의 완전한 구현을 제공합니다.

1.1 디렉토리 구조

```
x/evm/
├── client/ # CLI 명령어
├── config/ # EVM 설정
├── contracts/ # 사전 컴파일된 컨트랙트
├── keeper/ # 상태 관리자
├── precompiles/ # 사전 컴파일된 함수들
├── state/ # EVM 상태 관리
├── types/ # 타입 정의
├── abci.go # ABCI 인터페이스
├── autocli.go # 자동 CLI 생성
└── module.go # 모듈 정의
```

2. EVM Keeper 구조

2.1 Keeper 구조체

```
type Keeper struct {
ac address.Codec
cdc codec.Codec
storeService corestoretypes.KVStoreService

accountKeeper types.AccountKeeper
bankKeeper types.BankKeeper
communityPoolKeeper types.CommunityPoolKeeper
ibcHookKeeper types.IBCHookKeeper
gasPriceKeeper types.GasPriceKeeper
erc20Keeper types.IERC20Keeper
erc20StoresKeeper types.IERC20StoresKeeper
erc721Keeper types.IERC721Keeper
txUtils types.TxUtils

// grpc routers
msgRouter baseapp.MessageRouter
grpcRouter types.GRPCRouter

config evmconfig.EVMConfig
```

```
// the address capable of executing a MsgUpdateParams message
authority  string


Schema collections.Schema
Params collections.Item[types.Params]
VMStore collections.Map[[]byte, []byte]


// 컨트랙트 주소들
ERC20FactoryAddr collections.Item[[]byte]
ERC20WrapperAddr collections.Item[[]byte]
ConnectOracleAddr collections.Item[[]byte]


// ERC20 관련 저장소
ERC20s collections.KeySet[[]byte]
ERC20Stores collections.KeySet[collections.Pair[[]byte, []byte]]
ERC20DenomsByContractAddr collections.Map[[]byte, string]
ERC20ContractAddrsByDenom collections.Map[string, []byte]


// ERC721 관련 저장소
ERC721ClassURIs collections.Map[[]byte, string]
ERC721ClassIdsByContractAddr collections.Map[[]byte, string]
ERC721ContractAddrsByClassId collections.Map[string, []byte]
```

```
// EVM 블록 해시 저장소
EVMBlockHashes collections.Map[uint64, []byte]


queryCosmosWhitelist types.QueryCosmosWhitelist
cosmosCallbackABI *abi.ABI
}
```

2.2 Keeper 초기화

```
func  NewKeeper(
ac address.Codec,
cdc codec.Codec,
storeService corestoretypes.KVStoreService,
accountKeeper types.AccountKeeper,
bankKeeper types.BankKeeper,
communityPoolKeeper types.CommunityPoolKeeper,
ibcHookKeeper types.IBCHookKeeper,
gasPriceKeeper types.GasPriceKeeper,
msgRouter baseapp.MessageRouter,
grpcRouter *baseapp.GRPCQueryRouter,
authority  string,
evmConfig evmconfig.EVMConfig,
queryCosmosWhitelist types.QueryCosmosWhitelist,
```

```
) *Keeper
```

3. EVM 상태 관리

3.1 VMStore

EVM의 모든 상태 데이터를 저장하는 키-값 저장소:

```
VMStore  collections.Map[[]byte, []byte]
```

3.2 블록 해시 추적

```
func (k Keeper) TrackBlockHash(ctx context.Context, blockHeight  uint64, hash co

num, err := k.NumRetainBlockHashes(ctx)

if  err != nil {

return  err

}


// keep all block hashes if `NumRetainBlockHashes` is 0

if  num != 0 && blockHeight > num {

err := k.EVMBlockHashes.Remove(ctx, blockHeight-num)

if  err != nil {

return  err

}

}
```

```
return  k.EVMBlockHashes.Set(ctx, blockHeight, hash.Bytes())
}
```

4. ERC20 토큰 브리징

4.1 ERC20 Keeper 구조

```
type IERC20Keeper interface {
// 토큰 생성 및 관리
CreateERC20(ctx context.Context, denom  string) (common.Address, error)
// 토큰 변환
ConvertCoinToERC20(ctx context.Context, sender sdk.AccAddress, receiver common.A
ConvertERC20ToCoin(ctx context.Context, sender common.Address, receiver sdk.AccA
// 메타데이터 관리
SetERC20Metadata(ctx context.Context, contractAddr common.Address, metadata bank
}
```

4.2 토큰 변환 프로세스

Cosmos Coin → ERC20

1. 사용자가 Cosmos 네이티브 토큰을 ERC20으로 변환 요청
2. `ConvertCoinToERC20` 함수 호출

3. 네이티브 토큰을 모듈 계정으로 전송

4. 해당하는 ERC20 토큰을 사용자에게 민팅

ERC20 → Cosmos Coin

1. 사용자가 ERC20 토큰을 Cosmos 네이티브 토큰으로 변환 요청

2. `ConvertERC20ToCoin` 함수 호출

3. ERC20 토큰을 번

4. 해당하는 네이티브 토큰을 사용자에게 전송

4.3 ERC20 저장소 관리

```
// ERC20 컨트랙트 등록
ERC20s  collections.KeySet[[]byte]

// ERC20 저장소 추적
ERC20Stores  collections.KeySet[collections.Pair[[]byte, []byte]]

// 컨트랙트 주소 ↔ Denom 매핑
ERC20DenomsByContractAddr  collections.Map[[]byte, string]
ERC20ContractAddrsByDenom  collections.Map[string, []byte]
```

5. ERC721 NFT 브리징

5.1 ERC721 Keeper 구조

```
type IERC721Keeper interface {
// NFT 컬렉션 생성
CreateERC721(ctx context.Context, classId  string, classUri  string, classData
// NFT 변환
ConvertNFTToERC721(ctx context.Context, sender sdk.AccAddress, receiver common.A
ConvertERC721ToNFT(ctx context.Context, sender common.Address, receiver sdk.AccA
}
```

5.2 NFT 저장소 관리

```
// 클래스 URI 저장
ERC721ClassURIs  collections.Map[[]byte, string]

// 컨트랙트 주소 ↔ 클래스 ID 매핑
ERC721ClassIdsByContractAddr  collections.Map[[]byte, string]
ERC721ContractAddrsByClassId  collections.Map[string, []byte]
```

6. PreBlock 처리

6.1 PreBlock 함수

```
func  PreBlock(ctx sdk.Context, k *keeper.Keeper) (sdk.ResponsePreBlock, error)
defer  telemetry.ModuleMeasureSince(types.ModuleName, time.Now(), telemetry.Metr
```

```
height := uint64(ctx.BlockHeight())
if  height <= 1 {
return sdk.ResponsePreBlock{}, nil
}


// current block header hash is created by the previous block, so we track it wi
return sdk.ResponsePreBlock{}, k.TrackBlockHash(ctx, height-1, common.BytesToHas
}
```

6.2 블록 해시 추적 목적

- EVM의 `BLOCKHASH` opcode 지원
- 최근 256개 블록 해시 유지 (Ethereum 표준)
- 스마트 컨트랙트에서 이전 블록 해시 참조 가능

7. 모듈 인터페이스

7.1 AppModule 구현

```
type AppModule struct {
AppModuleBasic
keeper *keeper.Keeper
}
```

```go
func (am AppModule) RegisterServices(cfg module.Configurator) {
types.RegisterMsgServer(cfg.MsgServer(), keeper.NewMsgServerImpl(am.keeper))
querier := keeper.NewQueryServer(am.keeper)
types.RegisterQueryServer(cfg.QueryServer(), querier)
}
```

7.2 PreBlocker 구현

```go
func (am AppModule) PreBlock(ctx context.Context) (appmodule.ResponsePreBlock, e
c := sdk.UnwrapSDKContext(ctx)
return  PreBlock(c, am.keeper)
}
```

8. 설정 관리

8.1 EVM 설정 구조

```go
type EVMConfig struct {
// 시뮬레이션 가스 제한
ContractSimulationGasLimit  uint64
// 인덱서 설정
IndexerDisable  bool
IndexerRetainHeight  uint64
```

```
IndexerBackfillStartHeight  uint64

// 기타 EVM 관련 설정들

}
```

8.2 설정 접근

```
func (k Keeper) Config() evmconfig.EVMConfig {

return  k.config

}
```

9. 트랜잭션 유틸리티

9.1 TxUtils 인터페이스

```
type TxUtils interface {

// 트랜잭션 생성 및 실행 유틸리티

BuildTx(ctx context.Context, msgs []sdk.Msg) ([]byte, error)

ExecuteTx(ctx context.Context, txBytes []byte) (*sdk.Result, error)

}
```

9.2 활용 사례

- EVM에서 Cosmos 트랜잭션 실행
- 크로스 체인 메시지 처리

- 자동화된 토큰 변환

10. 보안 및 권한 관리

10.1 Authority 관리

```
func (ak Keeper) GetAuthority() string {
return  ak.authority
}
```

10.2 Query Cosmos Whitelist

```
queryCosmosWhitelist  types.QueryCosmosWhitelist
```

- EVM에서 Cosmos 모듈 쿼리 시 보안 제어
- 허용된 쿼리만 실행 가능
- 악의적인 컨트랙트로부터 보호

이러한 구조를 통해 EVM 모듈은 Ethereum과 완전히 호환되면서도 Cosmos 생태계의 모든 기능을 활용할 수 있는 통합 환경을 제공합니다.